

Part 3: Prompt Preparation

Chosen PR: PR #196 = Added Separate Socket Groups to Client

Repository: aio-lib/aio-kafka

3.1.1 Repository Context

aio-kafka is a Python library used to work with Apache Kafka using asyncio and async/await. It helps Python applications send and receive Kafka messages without blocking the program execution. It is mainly used in backend systems, microservices, and real-time data streaming applications.

The library mainly provides:

- AIOKafkaProducer = sends messages
- AIOKafkaConsumer = reads messages

These work on top of AIOKafkaClient which manages broker connections and request handling.

The project involves concepts like async networking, consumer groups, offsets, and distributed systems communication.

3.1.2 Pull Request Description

PR #196 improves how aio-kafka manages broker connections. Earlier, the client used only one TCP socket per broker for all requests. Because of this, long fetch requests could block important coordination requests like heartbeats and offset commits.

If heartbeats are delayed too long, Kafka may think the consumer has failed and start a rebalance process.

To solve this problem, the PR adds two socket groups:

- DEFAULT = used for normal requests like Fetch and Produce
- COORDINATION = used for heartbeat and group management requests

This allows coordination requests to run separately without waiting for fetch requests.

3.1.3 Acceptance Criteria

The implementation should

- Create separate connections for different socket groups

- Allow heartbeat requests to work even during long fetch requests
- Use the COORDINATION group for all group management requests
- Keep old behavior unchanged for normal requests
- Prevent one socket failure from affecting another socket group
- Pass all existing tests successfully

3.1.4 Edge Cases

Same broker handling multiple roles

One broker may handle both data requests and coordination requests, so separate sockets must still work correctly.

Consumer without group_id=If no consumer group is used, the COORDINATION socket should not be created.

Connection failure=If one socket group fails, the other group should continue working normally.

Rebalance during fetch=Heartbeat and rebalance requests should continue working even if a fetch request is blocked.

3.1.5 Initial Prompt

The task is to add socket groups in aiokafka so that different request types use separate TCP connections. The DEFAULT group will handle normal requests while the COORDINATION group will handle heartbeat and consumer group requests.

The main files to update are:

- client.py = manage separate socket connections
- group_coordinator.py = send coordination requests using the new group
- conn.py = update connection details if needed
- test_client.py =add tests for socket separation

The implementation should maintain backward compatibility and ensure that both socket groups work independently without affecting each other.

I declare that all written content in this assessment is my own work, created without the use of AI language models or automated writing tools. All technical analysis and documentation reflects my personal understanding and has been written in my own words.